Machine Learning with Graphs

Beyond Weisfeiler–Lehman Persistent and Cycle-Aware Graph Representations

A Comprehensive Case Study Report

Student Name: _____

Roll Number: _____

Department of Computer Science and Engineering

Amrita Vishwa Vidyapeetham

Academic Year 2026–2027

Abstract

Graph classification assigns a label to an entire graph, yet many useful distinctions depend on both local neighborhoods and global connectivity. The Weisfeiler–Lehman (WL) subtree representation efficiently counts iteratively refined node labels, but its neighborhood aggregation can fail to distinguish some graphs with different global structure. This case study implements standard WL and the persistent WL variants of Rieck, Bock, and Borgwardt, including component-persistence features (P-WL) and cycle-aware features (P-WL-C). It then evaluates two project adaptations: a scalar weight on the cycle block and a binary encoding of aggregated cycle-feature presence. The methods were assessed on MUTAG (188 molecular graphs) and PTC-MR (344 graphs) using ten repetitions of stratified ten-fold outer cross-validation with five-fold inner selection. The established methods and λ experiment were completed on both datasets; binary cycle encoding was evaluated on MUTAG only. On MUTAG, WL obtained 88.19 ± 1.23% accuracy and 86.70 ± 1.46% macro-F1, the highest means. P-WL-C improved over P-WL by 1.61 percentage points in accuracy, but remained below WL. Positive λ values produced identical predictions across the 100 tested folds; λ=0 reproduced P-WL. Binary encoding reduced P-WL-C accuracy by 1.72 points and macro-F1 by 1.98 points, while changing predictions on 67 of 100 folds. On PTC-MR, P-WL had the highest mean accuracy and macro-F1, and P-WL-C was lower than P-WL. These results show dataset-dependent effects rather than a universal advantage for persistence or cycles. The negative binary result remains a valid measured outcome of a custom representation modification.

Keywords: graph classification; Weisfeiler–Lehman refinement; persistent homology; cycle features; nested cross-validation; molecular graphs

Contents

Abstract ····· 2
1 Introduction ····· 4
2 Literature Review ····· 6
3 Theoretical Background ····· 7
4 Proposed Methodology ····· 10
5 Custom Modifications ····· 12
6 Dataset Description ····· 14
7 Experimental Setup ····· 15
8 Results and Analysis ····· 17
9 Discussion ····· 22
10 Limitations and Future Work ····· 24
11 Conclusion ····· 25
References ····· 26

List of Figures

1 Uniformly labelled graphs with different global connectivity
2 Distinct neighborhoods under one refinement step
3 Edge insertion, component merging, and cycle birth
4 Implemented processing and evaluation pipeline
5 MUTAG method comparison by accuracy
6 MUTAG method comparison by macro-F1
7 PTC-MR baseline comparison
8 MUTAG λ sensitivity
9 PTC-MR λ sensitivity
10 MUTAG magnitude versus binary cycle encoding
11 Paired fold differences for binary encoding

List of Tables

1 Notation used in the report
2 Dataset statistics verified from original archives
3 Fixed Random Forest and search configuration
4 MUTAG nested-CV results
5 PTC-MR nested-CV results
6 Paired MUTAG binary-encoding outcomes

1 Introduction

Graphs represent entities and relations jointly. A vertex may denote an atom, user, or component; an edge records a bond or interaction. The graph label depends not only on how many vertices of each kind occur, but also on which vertices connect and how local patterns combine into larger structures. Graph classification therefore requires a representation that turns a variable-size relational object into a form suitable for a classifier without discarding all meaningful structure.

Molecular graphs make this problem concrete. Atoms carry categorical labels and bonds define connectivity. A graph-level target such as mutagenicity or carcinogenicity can depend on substructures, but also on where those substructures sit in the molecule. A bag of atom counts ignores adjacency. A neighborhood-based representation preserves local context more effectively, while potentially compressing distinct global arrangements into the same feature counts.

The Weisfeiler–Lehman (WL) subtree procedure is a computationally efficient way to build graph features from iterative neighborhood refinement [1]. At each round, a vertex receives a signature determined by its previous signature and the multiset of neighboring signatures. Counting the resulting signatures at all depths gives an explicit graph vector. Under an inner product, those vectors define a graph kernel. The project uses the explicit vector representation with Random Forest (RF) classifiers rather than constructing a kernel matrix.

WL refinement is powerful, but local aggregation has limits. Some non-isomorphic graphs can retain the same multiset of refined labels at every iteration. A uniformly labelled six-cycle and two disjoint triangles illustrate a simple global difference: both are 2-regular and share the same uniform local refinement counts, while their component and cycle structure differs. This example motivates investigating topological summaries alongside local signatures; it does not imply that WL fails to detect every cycle or every global distinction.

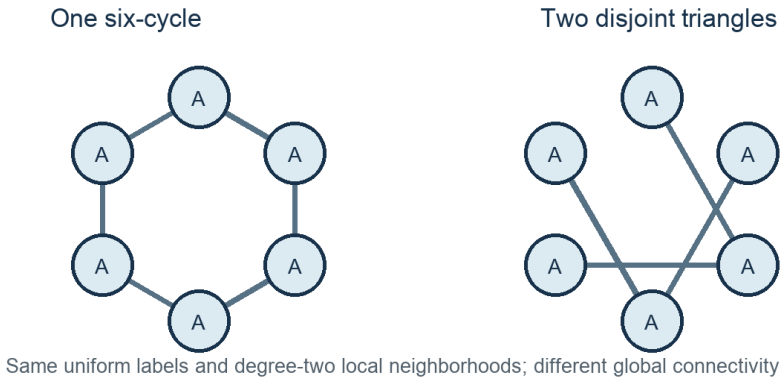


Figure 1. Uniformly labelled graphs with different global connectivity. Original schematic based on the WL limitation example checked in the project.

Persistent WL (P-WL) augments refined vertex patterns with component persistence under a weighted edge filtration. The cycle-aware P-WL-C variant additionally summarizes cycle births [2]. This report studies those established representations and two project-specific adaptations: (i) scaling the cycle block by λ and (ii) replacing its nonzero aggregate values by binary presence indicators. The central question is empirical: do these representations change nested-CV classification on two small molecular benchmarks, and in what direction?

1.1 Objectives and scope

The study has four objectives: implement WL, P-WL, and P-WL-C from their published definitions and author-source conventions; establish a repeatable nested evaluation on MUTAG and PTC-MR; test λ-weighting and binary cycle presence as controlled representation adaptations; and report both positive and negative outcomes without treating cross-validation folds as independent significance tests. The project is a documented reimplementation with protocol deviations, not an exact reproduction of the authors' published scores.

The completed evidence comprises 100 outer-fold evaluations per established method and per λ setting on each dataset, plus 100 MUTAG folds for P-WL-C-Binary. The binary variant was evaluated only on MUTAG. PTC-MR contributes supplementary baseline and λ comparisons, not a binary-cycle experiment. The report distinguishes method definitions from our changes, and measured observations from mechanistic hypotheses.

2 Literature Review

2.1 Weisfeiler–Lehman graph kernels

Shervashidze et al. introduced efficient graph kernels based on the Weisfeiler–Lehman graph isomorphism test [1]. The motivation is to extend the expressive power of simple graph kernels while retaining a computationally practical feature construction. Starting from categorical vertex labels, each refinement round encodes a vertex's previous label together with a sorted multiset of its neighbors' labels. Across rounds, increasingly large rooted neighborhoods influence the signature.

The WL subtree kernel compares graphs by counting matching refined labels at each iteration. Its explicit feature map has one coordinate for each rooted subtree pattern and iteration, and the kernel is the dot product of the concatenated count vectors. This view is useful in the present study because the project can supply the feature dictionaries directly to an RF, inspect feature dimensions, and fit the feature vocabulary within each training partition. It also makes clear what is represented: multiset summaries of neighborhoods, not a complete encoding of arbitrary graph topology.

The paper formalizes the relationship between refinement and graph kernels and establishes strong empirical performance on graph classification tasks. It also discusses the distinction between inductive and transductive feature construction. This project adopts an inductive vocabulary: signatures are graph-local and the vectorizer is fitted only on training graphs in each inner or outer split. Thus a validation or test graph cannot introduce a feature column into the forest's fitted representation.

2.2 Persistent Weisfeiler–Lehman

Rieck, Bock, and Borgwardt proposed a persistent extension that couples WL labels with topological persistence [2]. Their method assigns weights to existing edges using label differences and differences between neighborhood label-count vectors. Edges are inserted in increasing weight order into a filtration that begins with all vertices. Connected-component persistence captures the scale at which components merge; cycles appear when an inserted edge connects vertices that are already connected.

P-WL aggregates persistence contributions by refined vertex label. The cycle-enhanced method P-WL-C adds a separate label-indexed block for cycle births. This construction is motivated by the fact that graph topology evolves over a filtration: a cycle's birth edge has a

weight that describes the scale at which the cycle becomes present. Unlike a persistent image, the implemented features remain explicit sums grouped by WL labels and refinement depth.

The paper reports improvements on several graph datasets, but those results are not a guarantee for every data distribution or classifier. The present work does not claim exact reproduction. It caps essential H0 components at the maximum edge weight as in the inspected author implementation, retains stable source-edge order to settle ties, uses h=0…6 rather than the paper's h=0…10, uses a common modern RF configuration, avoids scaling, and learns vocabularies per training split. Published table values are therefore context rather than directly comparable targets.

2.3 Attribution of the project contributions

WL is the foundational representation from Shervashidze et al. P-WL and P-WL-C are established method variants from Rieck et al.; P-WL-C is not presented as a novel contribution of this project. The project's adaptations are the λ-weighted concatenation of the cycle block and the binary transformation of completed cycle-feature coordinates. λ-weighting was empirically ineffective for positive values under this RF pipeline, while binary encoding reduced the MUTAG mean metrics. The scientific contribution is the measured implementation and its transparent negative result, not an asserted performance improvement.

The authors' source code is retained in the repository references directory to resolve implementation details that are underspecified in prose, including stable sorting and component-root attribution. Project notes document these decisions and departures. Both the mathematical paper and source repository inform implementation, but source-specific choices are explicitly reported so readers can distinguish the paper's method from this experimental protocol [2, 3].

3 Theoretical Background

3.1 Graph representation and classification

A finite attributed graph is written $G=(V,E,\ell,y)$, where V is a finite vertex set, E is a set of undirected edges, $\ell:V\to\Sigma$ assigns a categorical node label, and y is the graph-level target. An edge has two endpoints and the molecular inputs also include categorical bond labels; the implemented feature maps use ℓ and graph topology but ignore those bond labels. A graph classifier is a learned map $f:G\to\mathcal{Y}$ from a graph to one of the class labels.

Because graphs have different numbers of vertices and no common coordinate system, a feature map Φ(G) must summarize graph structure in a fixed sparse vector space. A dictionary representation maps named structural patterns to numeric values. A training-set vectorizer chooses the vocabulary and maps each dictionary to a sparse row. In this project, values can be counts, persistence-weighted sums, or binary cycle-presence indicators.

3.2 Weisfeiler–Lehman refinement

At depth zero, each vertex's state is its categorical input label. At each subsequent depth, the state records the previous state and a sorted multiset of the previous states among its neighbors. The implementation serializes that structured tuple and hashes it with SHA-256; the hash is only a deterministic identifier for a signature, while a collision registry checks that distinct serialized signatures do not share an identifier.

$$\ell_0(v) = \text{categorical node label of } v$$

$$\ell_h(v) = \mathrm{HASH}(\ell_{h-1}(v), \mathrm{SORT}(\{\ell_{h-1}(u) : u \in N(v)\})), \quad h \geq 1$$

For graph G and depth h, let n_h,a(G) be the number of vertices assigned refined label a at depth h. The depth-specific feature block records those counts under depth-namespaced keys. The cumulative vector concatenates all blocks from depth zero through H, preserving the depth identity of a pattern. This explicit map induces a positive-semidefinite kernel by an inner product:

$$\Phi\mathrm{WL}(G) = (n_0,a(G), n_1,a(G), \ldots, nH,a(G))a$$

$$\mathrm{kWL}(G,G') = \langle \Phi\mathrm{WL}(G), \Phi\mathrm{WL}(G') \rangle$$

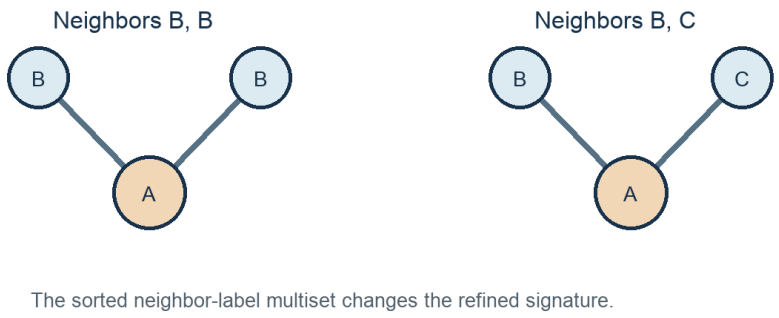


Figure 2. A refinement step distinguishes different neighbor-label multisets. Original schematic illustrating the update implemented in wl.py.

The multiset operation is essential: neighbor order is irrelevant, but multiplicity is retained. A node adjacent to labels B and B differs from one adjacent to B and C. Conversely, WL's

aggregation can map non-isomorphic graphs to identical feature counts; the six-cycle/two-triangles example in Figure 1 is one such limitation under uniform labels. WL is an efficiently computable refinement-based representation, not a complete graph-isomorphism test.

3.3 Persistent WL and the edge filtration

P-WL equips each existing edge with a positive weight. At depth zero, the weight is one plus an indicator of whether endpoint labels differ. At later depths, it adds one, the previous-label mismatch indicator, and the Minkowski p-distance between the endpoints' neighbor-only previous-label count vectors. If A and B are those count vectors, the last term is $(\Sigma_i|A_i-B_i|^p)^{1/p}$. The tested p values are 1 and 2; p also determines the power used in the persistence feature sum.

$$w_0(u,v) = 1 + \mathbb{1}[\ell_0(u) \neq \ell_0(v)]$$

$$w_h(u,v) = 1 + \mathbb{1}[\ell_{h-1}(u) \neq \ell_{h-1}(v)] + \|A_h(u) - A_h(v)\|_p$$

For each depth, construct a filtration Gτ by including every vertex at filtration value zero and adding only the graph's existing edges in stable ascending weight order. An edge that joins two components causes a merge; one component dies at that edge's weight. An edge whose endpoints are already connected creates a cycle, corresponding to an H1 birth in this one-dimensional graph filtration. No clique complex or triangle filling is added. Surviving H0 components are assigned a finite cap equal to the maximum edge weight, or 1 for an edgeless graph.

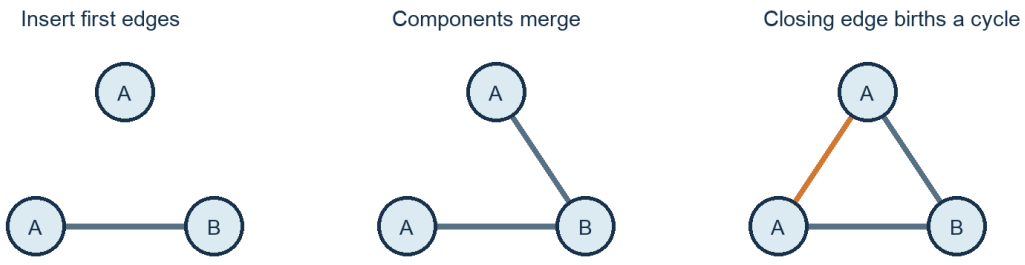


Figure 3. Edge insertion creates component merges and a cycle-closing edge. Original schematic of the graph filtration used by pwl.py; only existing edges are inserted.

For each refined label a at depth h, the component block sums the capped component death values raised to p for vertices with that label. The cycle block sums the cycle-birth edge weight raised to p for each endpoint label. A cycle-closing edge therefore contributes at both

endpoints; if both endpoints share label a, that label receives two contributions. The blocks remain separate and are namespaced by depth and label.

$$V_h(a) = \Sigma_{v:\ell_h(v)=a}\ \mathrm{death}(v)^p$$

$$C_h(a) = \Sigma_{e=(u,v)\ \text{closing a cycle}}\ w(e)^p\ [\mathbb{1}(\ell_h(u)=a)+\mathbb{1}(\ell_h(v)=a)]$$

3.4 Cycle information and feature representation

A cycle is a closed path. In the graph filtration, a cycle birth occurs when an edge closes a path already present in the inserted subgraph. Component persistence summarizes how graph pieces connect, whereas cycle features summarize redundant connectivity at the insertion scale. These are different topological events: tree edges merge components, while cycle-closing edges do not change the number of connected components.

The P-WL-C vector concatenates the V and C blocks over depths zero through H. The implementation retains a complete cycle alphabet for every current node-label alphabet, including zero-valued coordinates. This matters because the RF samples from the feature columns; dropping shared-zero columns would change the available feature set and would not be a neutral storage optimization. At $\lambda=0$ the cycle columns are physically omitted so the representation matches P-WL's vocabulary and feature-sampling dimension.

3.5 Notation

Symbol	Meaning
$G=(V,E,\ell,y)$	Attributed input graph and target label
h, H	Refinement depth and maximum depth
p	Minkowski exponent and persistence power, $p\in\{1,2\}$
$w_h(e)$	Edge filtration weight at depth h
$V_h(a)$	Component-persistence sum for label a
$C_h(a)$	Cycle-birth contribution for label a
λ	Scalar applied to the cycle block
$B_h(a)$	Indicator that aggregated $C_h(a)$ is nonzero

4 Proposed Methodology

The pipeline begins with the original TU molecular graph archives, applies iterative WL refinement, calculates persistent component and cycle features, constructs a method-specific dictionary, fits a training-local vectorizer, and trains an RF under nested cross-validation. The outer test fold is reserved for the final score of that split. The binary follow-up reads cached corrected persistence dictionaries and reuses the same stored MUTAG splits; it does not recompute filtrations.

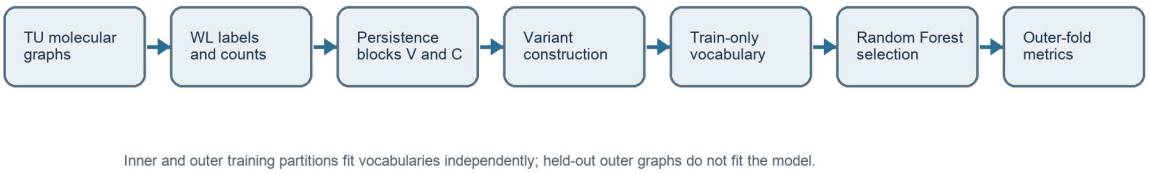


Figure 4. Implemented processing and evaluation pipeline. Workflow synthesized from data.py, wl.py, pwl.py, evaluate.py, and binary_cycle.py.

4.1 Loading and preprocessing

data.py loads the original TU archive and verifies it through PyTorch Geometric's TUDataset API (cleaned=False). It maps each sample into a NetworkX graph while preserving the original TU node and edge ordering needed for deterministic filtration tie handling. Assertions compare graph counts, node labels, targets, and edge sets against the raw archive. The archive identity and SHA-256 are stored with dataset metadata. All graphs in these molecular datasets are nonempty; the source checks also verify class counts and within-graph edges.

Node labels are categorical inputs; edge labels are available in the source data but are ignored by all evaluated feature maps. No node or edge continuous attributes are present in these archive metadata. The graph target is encoded internally as two classes, while reported class distributions retain the original -1 and +1 labels. These details restrict the interpretation of the results: they measure node-label and topology representations, not bond-label-aware modeling.

4.2 Feature construction and classifier fitting

wl.py computes graph-local refinement signatures and label-count dictionaries through H=6. pwl.py builds per-depth edge weights and uses a stable union-find procedure to record component deaths and cycle-closing edge indices. For each training split, evaluate.py fits DictVectorizer only on the training rows and transforms held-out rows against that vocabulary. Test-only structural signatures are ignored by the trained model. The RF consumes sparse explicit features, so no graph Gram matrix or global feature scaling is needed.

The binary transform operates after cycle contributions have been aggregated by depth and label. It replaces each cycle coordinate c by $1[c \neq 0]$, leaves the component block unchanged, retains the same feature keys and ordering, and does not mutate source caches. Since it needs

no new persistence calculation, the follow-up reuses the exact corrected P-WL-C caches from MUTAG. The implementation includes checks for column dimensions, vertex-block preservation, and cache immutability.

4.3 Nested selection and evaluation

Ten repetitions of stratified ten-fold outer cross-validation produce 100 saved outer predictions per completed method and dataset. Within each outer training partition, stratified five-fold inner cross-validation selects hyperparameters using mean validation accuracy. The outer test fold is not used to choose h, p, the number of trees, or λ. The saved split definitions are shared across methods, making paired fold comparisons possible.

Candidate settings include h=0…6, p∈{1,2}, and RF sizes 25, 50, or 100 trees. The λ extension additionally searches {0,0.25,0.5,1,2}. The binary follow-up uses the same depth, p, and tree grid without an additional λ search: 7×2×3=42 candidates per outer split. Deterministic ties prefer smaller h, then smaller p, tree count, and λ. Accuracy and macro-F1 are computed on the selected outer predictions with zero_division=0 for the latter.

The evaluation code can reuse a single fitted 100-tree forest's ordered estimator prefixes to obtain the 25- and 50-tree predictions. Project scientific checks compared these prefixes with independent smaller fits. This is a computation strategy; it does not reduce the set of candidate tree counts or change the validation protocol. Every fold and its candidate-selection records are saved in CSV and checkpoint JSON files for auditability.

5 Custom Modifications

5.1 Lambda-weighted cycle features

The first adaptation separates representation inclusion from representation magnitude. Let V(G) denote the concatenated component block and C(G) the concatenated cycle block. The λ experiment forms Φλ(G)=[V(G), λC(G)] for λ in {0,0.25,0.5,1,2}. At λ=0, the implementation omits C columns entirely. This yields exactly the P-WL representation and preserves its vocabulary size, unlike retaining a block of zero columns, which would change RF feature subsampling.

$$\Phi\lambda(G) = [V(G), \lambda\, C(G)] \; ; \; \lambda \in \{0, 0.25, 0.5, 1, 2\}$$

For a positive λ, the coordinate transformation is a positive scalar multiplication of cycle columns. Axis-aligned decision-tree splits depend on ordered values, so exact arithmetic

preserves their ordering and can preserve available partitions. The tested RF pipeline showed identical predictions for λ=0.25, 0.5, 1, and 2 relative to P-WL-C on all 100 MUTAG outer folds and all 100 PTC-MR outer folds. λ=0 matched P-WL exactly. This is a measured property of these datasets, saved splits, code, numerical implementation, and RF setup; it is not a theorem of invariance for every model, dataset, or computation.

The nested selector chose λ=0.25 on 95 of 100 MUTAG folds and λ=0 on five; for PTC-MR it chose 0.25 on 72 and zero on 28. Since every tested positive value ties at the prediction level, the smallest positive value wins deterministic exact ties. It should not be described as an empirically optimal cycle magnitude. The λ-selected summary can differ slightly from fixed P-WL-C because inner selection sometimes chooses the no-cycle option and because the reported average aggregates repeated fold-level selections.

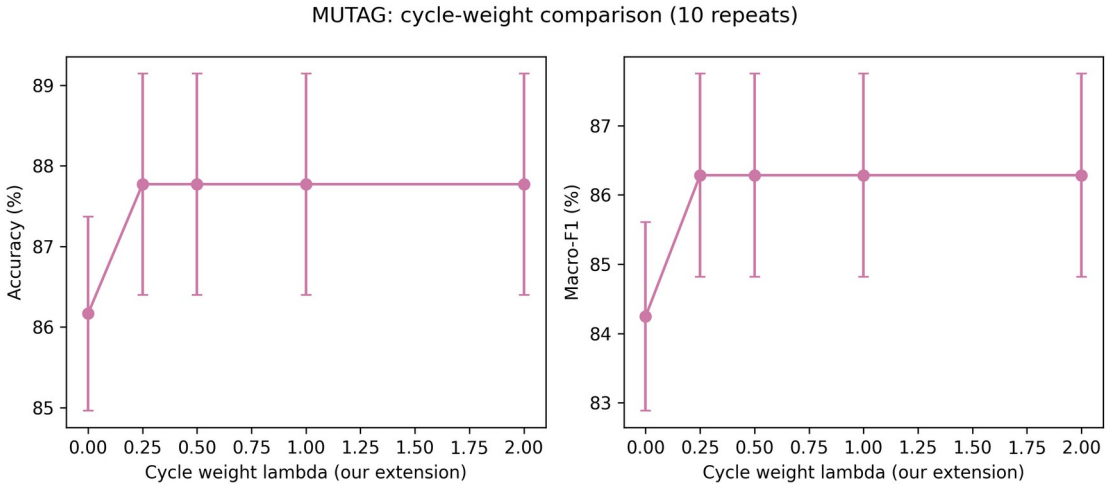


Figure 8. MUTAG λ sensitivity from the completed full run. Source: results/full/MUTAG_lambda.png; all tested positive λ settings have matching predictions.

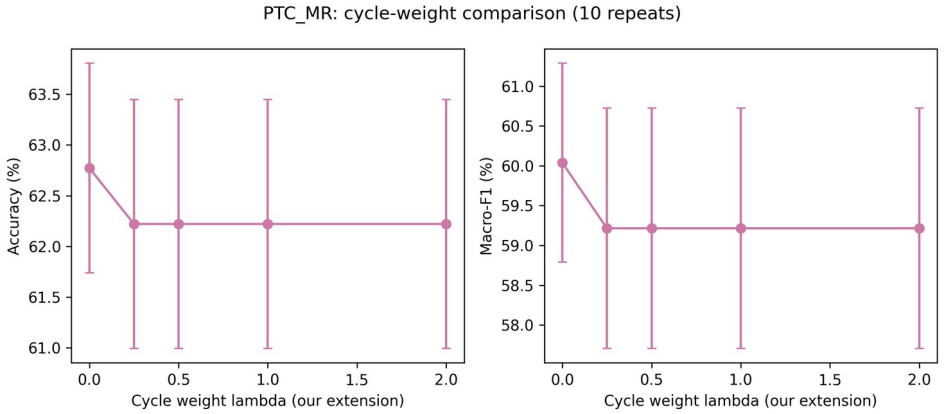


Figure 9. PTC-MR λ sensitivity from the completed full run. Source: results/full/PTC_MR_lambda.png; matching predictions are verified for the 100 tested folds.

5.2 Binary cycle-presence encoding

The primary representation modification asks whether the presence of a refined-label cycle contribution can be more useful than its magnitude. After $C_h(a)$ has been aggregated over cycle-closing edges, the binary variant maps it to $B_h(a)=1[C_h(a)\neq 0]$. It does not binarize individual edges or alter vertex persistence. A cycle coordinate with any positive aggregate value becomes one; a zero coordinate stays zero. This changes the information content by collapsing all positive magnitudes for a coordinate.

$$B_h(a) = \mathbb{1}[C_h(a) \neq 0] \; ; \quad \Phi\text{binary}(G) = [V(G), B(G)]$$

This modification is project-specific and distinct from the established P-WL-C definition. The original magnitude representation encodes both presence and weight; the binary form retains presence while discarding that magnitude. MUTAG was the primary modification dataset. Binary encoding was evaluated there only as the follow-up experiment; PTC-MR was used for supplementary baseline and λ comparisons, and binary was not tested on PTC-MR. The scope choice does not support a claim of generalization across datasets.

The hypothesis was that a simpler presence signal might regularize noisy magnitude values. The observed results did not support a performance gain on MUTAG: mean accuracy declined by 1.72 percentage points and macro-F1 by 1.98 points relative to P-WL-C. Predictions changed in 67 folds and selected parameters in 96 folds, so this was not a no-op. These are valid experimental findings: a custom modification is defined by a tested change to the method, not by whether its scores increase. The evidence supports implementation and measured effect, while the effect observed here is negative.

6 Dataset Description

MUTAG and PTC-MR are small molecular graph classification benchmarks from the TU collection [4]. Each graph represents a molecule, with atoms as vertices and bonds as edges. The provided archives contain categorical node and edge labels, but no continuous node or edge attributes. Project metadata was recomputed from the local original archives and records archive hashes. The implemented WL and persistent feature pipelines consume atom labels and graph topology; bond labels are explicitly ignored.

Statistic	MUTAG	PTC-MR
Graphs	188	344
Target classes	2 (−1, +1)	2 (−1, +1)
Class counts (−1, +1)	63, 125	192, 152
Total nodes	3,371	4,915

Statistic	MUTAG	PTC-MR
Undirected edges	3,721	5,054
Distinct node-label types	7	18
Distinct edge-label types (unused)	4	4
Mean nodes per graph	17.9309	14.2878
Mean edges per graph	19.7926	14.6919

Table 2. Dataset statistics computed from the archived TU data. Sources: results/binary-cycle/dataset_statistics.json and results/full/*_data.json.

6.1 MUTAG

MUTAG contains 188 molecular graphs with a binary mutagenicity target. The original class labels are imbalanced: 63 graphs are labeled −1 and 125 are labeled +1. The graphs contain 3,371 atoms and 3,721 undirected bonds in total, with seven distinct atom-label categories and four bond-label categories. The mean graph has 17.93 nodes and 19.79 edges. MUTAG is the primary dataset for binary-cycle analysis because the complete corrected baseline and same-fold binary follow-up were saved for 100 outer folds.

6.2 PTC-MR

PTC-MR contains 344 molecular graphs for male-rat carcinogenicity classification. It has 192 graphs in class −1 and 152 in class +1, 4,915 nodes, 5,054 undirected edges, 18 node-label types, and four edge-label types. Average size is 14.29 nodes and 14.69 edges per graph. It serves as a supplementary dataset for the established methods and λ experiment. P-WL-C-Binary was not run on PTC-MR; no result, missing-fold count, or cross-dataset binary comparison should be inferred for that method.

The small sample counts and class imbalance make fold-level variability relevant, and repeated cross-validation reuses the same molecular graphs. Reported repeat dispersion is therefore descriptive. These benchmarks provide a controlled comparison rather than a broad basis for claims about chemical activity prediction in deployment settings.

7 Experimental Setup

7.1 Search space and classifier

The completed corrected full runs use ten repeats, h=0…6, p∈{1,2}, and RF tree counts {25,50,100}. The λ method searches {0,0.25,0.5,1,2}. P-WL-C-Binary uses 42 depth–p–tree candidates per fold and no λ grid. The fixed RF configuration uses the Gini criterion, square-

root feature subsampling, bootstrap sampling, unlimited maximum depth, minimum split size 2, minimum leaf size 1, no class weighting, random_state 42, and one job per forest. Eight workers evaluate independent outer folds.

Setting	Value
Refinement depth	h = 0, 1, 2, 3, 4, 5, 6
Persistence exponent	p ∈ {1, 2}
RF estimators	25, 50, 100
λ grid	0, 0.25, 0.5, 1, 2
Outer protocol	10 repeats × stratified 10-fold = 100 folds
Inner protocol	Stratified 5-fold within each outer training set
Selection metric	Mean inner validation accuracy
Reported metrics	Outer accuracy and macro-F1
RF configuration	Gini; sqrt features; bootstrap; seed 42; unweighted

Table 3. Fixed model and search settings from results/full/config.json, evaluate.py, and results/binary-cycle/config.json.

7.2 Cross-validation and leakage control

For each repeat r, the outer stratified folds use random_state=r. Inner folds use random_state=1000+10r+f for outer fold f. Each inner train/validation split is contained in the outer training set, and the outer test indices are disjoint. Every method uses the same saved splits. This design estimates performance after model selection while reserving the outer fold from candidate selection. It does not create 100 independent datasets: graphs recur in different folds and repeats.

Each inner and outer training partition fits its own DictVectorizer vocabulary. Held-out features absent from that training vocabulary are ignored. No scaler is fit over validation or test data; RF split rules do not require standardization. These decisions avoid the leakage mode documented in the inspected author scripts, which scale the entire outer training set before inner validation. They also make the λ transformation interpretable as direct cycle-coordinate scaling rather than a post-scaling artifact.

7.3 Metrics and uncertainty

Accuracy is the proportion of correctly classified graphs in an outer test fold. Macro-F1 is the arithmetic mean of the class-wise F1 scores, giving the two classes equal weight. Within each repeat, the ten outer-fold metric values are first averaged. The reported mean is then the arithmetic mean of ten repeat means; the reported standard deviation is the population standard deviation (ddof=0) across those ten means. Percentage values are obtained by multiplying the saved fractional metrics by 100.

$$\mu = (1/10)\,\Sigma_r\, m_r\,; \quad \sigma = \sqrt{[(1/10)\,\Sigma_r\,(m_r - \mu)^2]}$$

This standard deviation describes repeat-to-repeat variability under the saved repeated-fold protocol. It is not a confidence interval, standard error, or formal significance test. Since repeated folds reuse the same observations, no independent-fold inference is claimed. Paired binary comparisons are reported as descriptive differences on identical saved folds, with win/loss/tie counts and repeat-level difference dispersion.

7.4 Software, provenance, and resources

The README records testing with Python 3.9.6 on macOS arm64 and pinned package versions: NumPy 2.0.2, SciPy 1.13.1, scikit-learn 1.6.1, NetworkX 3.2.1, GUDHI 3.11.0, Matplotlib 3.9.4, pandas 2.3.3, Requests 2.32.5, urllib3 1.26.20, PyTorch 2.8.0, PyTorch Geometric 2.6.1, and Pillow 11.3.0. The run metadata records eight workers. The available project records do not identify a specific CPU model, RAM capacity, or GPU; this report does not infer those resources.

Feature dictionaries are cached by source and configuration. Corrected baseline CSVs and checkpoints are preserved under results/full; the binary follow-up is isolated under results/binary-cycle. A preservation record reports that all 712 pre-existing result/cache files and original method functions remained unchanged during the follow-up. The saved manifests, fold predictions, candidate summaries, and verification records provide the provenance used for this report.

8 Results and Analysis

8.1 MUTAG baseline and modification results

Table 4 gives the nested-selected method estimates. WL achieved the highest mean accuracy and macro-F1. P-WL-C improved over P-WL by 1.60526 percentage points in accuracy and 2.03762 points in macro-F1, yet remained below WL. λ-selected rounded to the same values as P-WL-C, while its repeat dispersion differed slightly. Binary encoding fell below P-WL-C by 1.71637 accuracy points and 1.98133 macro-F1 points.

Method	Accuracy (%)	Macro-F1 (%)
WL	88.19 ± 1.23	86.70 ± 1.46
P-WL	86.17 ± 1.20	84.25 ± 1.36
P-WL-C	87.77 ± 1.37	86.28 ± 1.47
λ-selected	87.77 ± 1.26	86.28 ± 1.35
P-WL-C-Binary	86.06 ± 1.16	84.30 ± 1.60

Table 4. MUTAG nested-CV estimates. Mean ± population SD over ten repeat means. Source: results/full/MUTAG_summary.csv and results/binary-cycle/MUTAG_summary.csv; selected rows only.

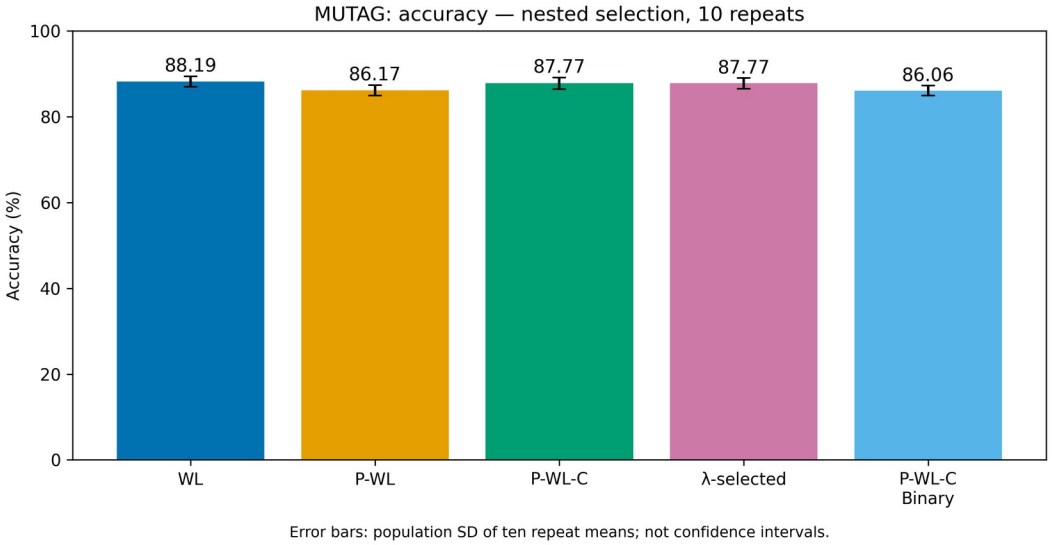


Figure 5. MUTAG mean accuracy and repeat variability by method. Source: results/binary-cycle/figures/MUTAG_accuracy_comparison.png.

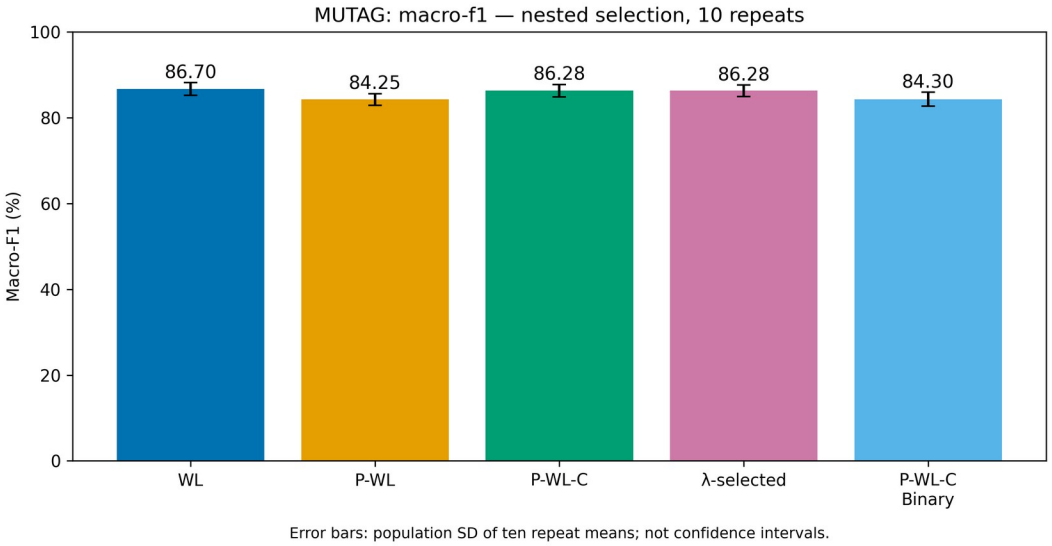


Figure 6. MUTAG mean macro-F1 and repeat variability by method. Source: results/binary-cycle/figures/MUTAG_macro_f1_comparison.png.

The two plots visually reinforce the table: WL leads in mean performance, and the binary variant is below magnitude P-WL-C on both metrics. Error bars represent population standard deviation across ten repeat means, not inferential uncertainty. Rounded error-bar overlap or non-overlap is not used to infer significance. The high-level conclusion is descriptive and tied to this dataset and protocol.

8.2 PTC-MR supplementary comparisons

Table 5 summarizes the established methods and λ-selected pipeline on PTC-MR. P-WL has the largest mean accuracy and macro-F1. P-WL-C is 0.55210 points lower than P-WL in accuracy and 0.82427 points lower in macro-F1. The cycle-feature effect therefore differs from MUTAG, where P-WL-C improved over P-WL but did not exceed WL. Binary encoding is not evaluated for this dataset.

Method	Accuracy (%)	Macro-F1 (%)
WL	62.33 ± 2.25	59.54 ± 2.47
P-WL	62.77 ± 1.03	60.04 ± 1.25
P-WL-C	62.22 ± 1.23	59.21 ± 1.51
λ-selected	62.49 ± 1.55	59.62 ± 1.87
P-WL-C-Binary	Not evaluated	Not evaluated

Table 5. PTC-MR completed nested-CV estimates. Binary cycle encoding was not evaluated. Source: results/full/PTC_MR_summary.csv.

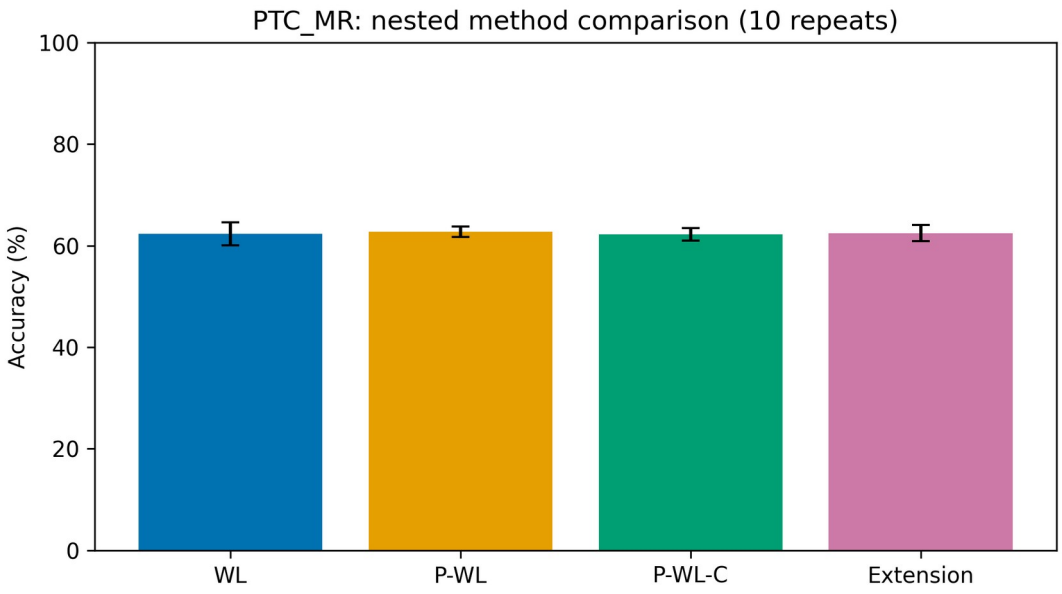


Figure 7. PTC-MR baseline ablation from the corrected full run. Source: results/full/PTC_MR_ablation.png.

The PTC-MR plot presents the four completed baseline/λ variants; it contains no binary result. Its pattern agrees with the saved summary: P-WL leads, while adding cycle magnitudes to P-WL is lower on the repeat-mean metrics. This is evidence against a universal cycle-feature gain in the tested setup, not evidence that cycles are uninformative in all molecular tasks.

8.3 Lambda weighting

The λ curves and saved fold audit agree: λ=0 exactly reproduces P-WL, and each positive tested λ setting yields matching predictions and selected h, p, and tree parameters with P-WL-C on all 100 outer folds per dataset. Thus positive magnitude scaling did not change the fitted RF's outputs in the tested folds. The explanation based on order-preserving feature scaling is plausible for axis-aligned trees, but the empirical statement is bounded to these 200 dataset-fold sets and this implementation.

The joint selector's zero choices explain why the λ-selected aggregate is not definitionally identical to P-WL-C. On MUTAG it selected zero in five folds; on PTC-MR, in 28. The smallest positive setting wins ties among positive settings because their predictions and inner accuracies tie under the saved procedure. Consequently the sweep tests whether cycles are included, but supplies little evidence about a useful positive magnitude level.

8.4 Binary paired-fold analysis

The same saved MUTAG folds permit paired comparisons between P-WL-C and P-WL-C-Binary. Accuracy improved in 17 folds, worsened in 41, and tied in 42. Macro-F1 improved in 18, worsened in 44, and tied in 38. Predictions differed on 67 of 100 folds, comprising 114 changed graph predictions among 1,880 test-prediction opportunities across the repeated folds. Selected h, p, and tree count changed on 96 folds; the parameter-specific counts were 83, 47, and 67, respectively.

Paired outcome	Accuracy	Macro-F1
Mean difference (binary − magnitude), pp	−1.71637	−1.98133
Improved folds	17	18
Worsened folds	41	44
Tied folds	42	38
Repeat-level difference SD, pp	1.79695	2.17208

Table 6. Paired MUTAG binary outcomes across the 100 identical outer folds. Source: results/binary-cycle/comparison.json and paired_folds.csv.

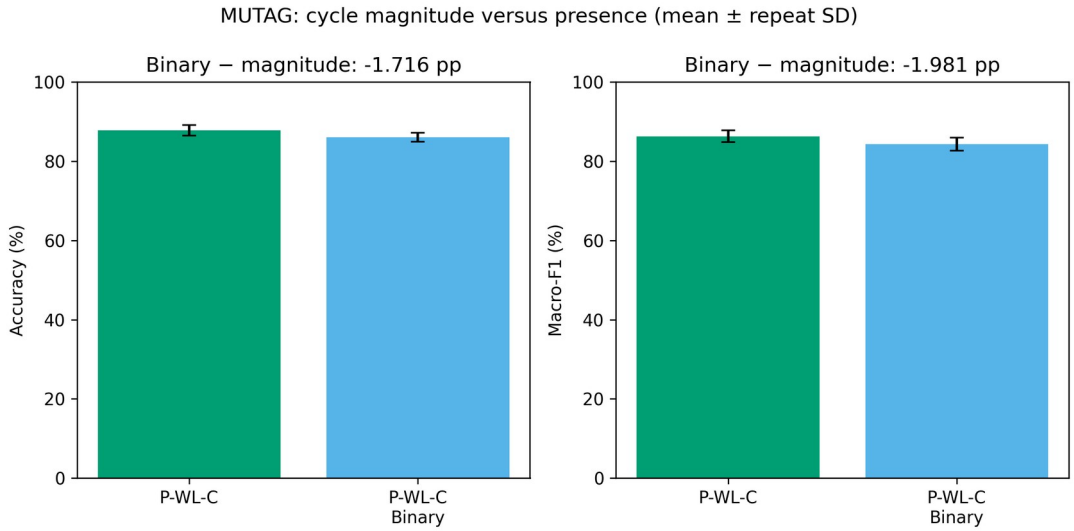


Figure 10. Magnitude P-WL-C versus binary cycle encoding on MUTAG. Source: results/binary-cycle/figures/MUTAG_cycle_binary_comparison.png.

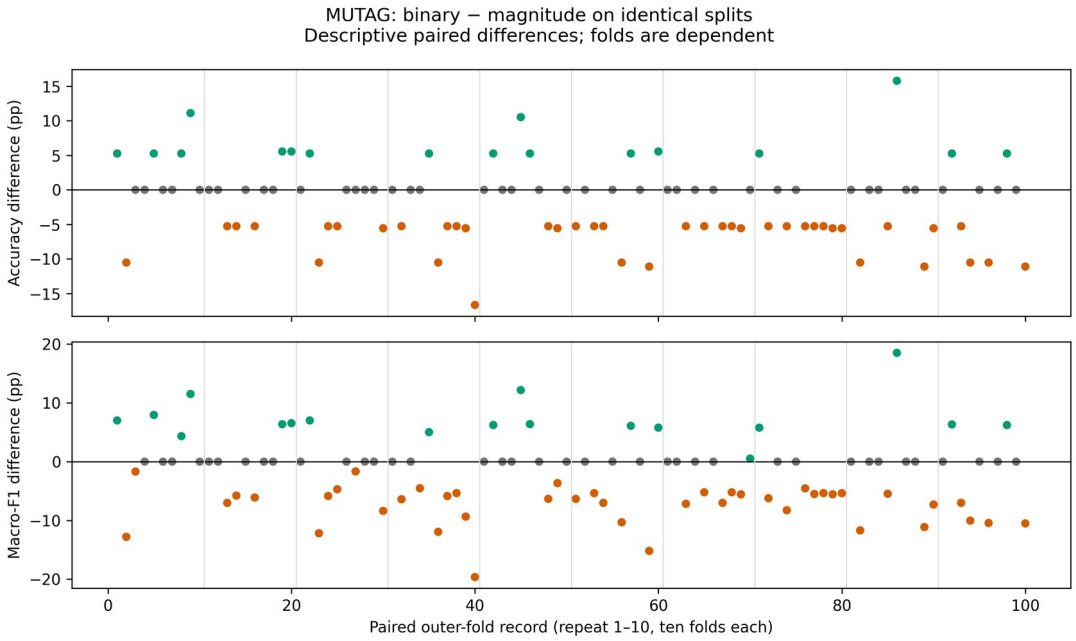


Figure 11. Paired outer-fold metric differences for binary minus magnitude features. Source: results/binary-cycle/figures/MUTAG_paired_fold_differences.png.

Binary encoding changes the representation and was empirically consequential, but its mean effect was negative. These differences compare two nested pipelines whose inner hyperparameter selection can choose different h, p, and tree counts; they are not a fixed-model intervention. The observed values do not establish that magnitude information caused the decline. They are consistent with the hypothesis that positive cycle magnitudes may contain useful signal, but that explanation remains untested causally.

8.5 Cross-dataset interpretation and score provenance

Across the completed methods, P-WL-C improved on P-WL on MUTAG but declined relative to P-WL on PTC-MR. Standard WL remained strongest by both reported means on MUTAG; P-WL was strongest on PTC-MR. This dataset dependence argues against a universal benefit claim. The two datasets are both small molecular benchmarks, and their target balance and node-label vocabularies differ; these results cannot isolate which difference explains the contrasting ranking.

The final results are sourced from the corrected full baseline records in results/full/MUTAG_folds.csv and results/full/PTC_MR_folds.csv, with summaries in the corresponding *_summary.csv files. Binary evidence comes from results/binary-cycle/MUTAG_folds.csv, MUTAG_summary.csv, paired_folds.csv, paired_repeat_means.csv, and comparison.json. The analysis verifier recomputed all saved baseline and binary metrics from targets and fold predictions, checked repeated aggregation, selection records and checkpoint agreement. Only setting=selected rows are used for final method means.

Historical smoke, preflight, stage, and compact-cycle outputs are development artifacts. In particular, results/compact-cycle-columns contains a superseded incomplete experiment whose compact cycle vocabulary omitted zero coordinates; its MUTAG cycle result around 88.20% and partial PTC-MR run are excluded. The final cycle comparisons use the corrected complete cycle alphabet, where positive cycle matrices retain a coordinate for every label. The immutable earlier baseline summary at results/full/RESULTS.md is not the final five-method report; the root RESULTS.md and results/binary-cycle/RESULTS.md contain the completed case study.

9 Discussion

WL’s performance shows that iterative neighborhood-count features are a strong baseline for these data. On MUTAG, WL exceeds the persistence-based variants by the reported means. Adding persistence and cycle information to P-WL raises both MUTAG metrics, which suggests that the additional features can complement component summaries within this nested RF pipeline. However, P-WL-C does not exceed WL. On PTC-MR, P-WL itself leads the completed method set, and adding cycle coordinates reduces the average metrics. These

observations support a dataset-dependent account rather than a monotone hierarchy of increasingly complex representations.

The λ experiment illustrates how a nominally continuous hyperparameter can have little practical effect for a tree ensemble. Positive scaling preserves order in exact arithmetic, leaving potential threshold partitions unchanged. The saved implementation produced identical predictions for all tested positive values across the 100 MUTAG folds and 100 PTC-MR folds. Still, representation dimension, floating-point ties, vocabulary order, and algorithmic details matter; the empirical audit cannot justify a universal invariance statement. More importantly, zero is an inclusion switch because the code removes cycle columns at $\lambda=0$.

The binary modification changes more than scale: it collapses all positive aggregate cycle values into a single indicator. It changed outputs on most folds and selected settings on 96 folds, but the mean metrics declined. One interpretation is that magnitude carries useful distinctions for MUTAG; another is that binary transformation interacts with the RF search in a way that changes selected tree structure. Since the model is retuned for each representation, and no ablation holds the selected parameters fixed, the experiment cannot distinguish these explanations. The negative result still answers the implemented research question for the tested setup.

A key methodological strength is preserving held-out boundaries. Each split uses a training-only vocabulary, inner selection is nested, and every method shares stored folds. This makes the paired comparison auditable and limits information leakage. At the same time, results inherit small-data uncertainty and repeated-fold dependence. Population SD summarizes repeat variability but does not validate that small differences are statistically reliable. No formal hypothesis test is reported.

The study is an implementation case study rather than an application deployment evaluation. It does not use bond labels, external validation sets, or additional chemical endpoints. Its findings concern the exact molecular graph archives, feature definitions, RF configuration, search grid, and saved nested folds. Any claim about real-world toxicity prediction, robustness to chemical distribution shift, or general graph domains would require additional evidence.

10 Limitations and Future Work

10.1 Limitations

The benchmark scope is limited to two small molecular datasets. The sample size is 188 graphs for MUTAG and 344 for PTC-MR, with class imbalance in both. Ten repeated cross-validation runs reuse the same graphs; folds are dependent, and mean ± population SD is descriptive rather than a confidence interval. The study does not perform formal significance testing or claim causal explanations for score differences.

Binary cycle presence was evaluated only on MUTAG. Therefore, the binary result cannot establish whether the transformation generalizes to PTC-MR or to graph datasets outside chemistry. PTC-MR supplies supplementary comparisons for WL, P-WL, P-WL-C, and λ only. The classifier is a Random Forest, so the observed invariance under positive scaling and the binary effect may not hold for distance-based, linear, or neural models.

The graph representation uses categorical node labels and topology but ignores available categorical edge labels. The depth range $h \leq 6$ differs from the cited paper's $h \leq 10$. Fixed modern scikit-learn RF defaults, training-only vocabularies, no scaling, and source-order tie handling are documented adaptations; exact published score reproduction is not claimed. Since model selection is repeated separately for each representation, the paired binary metric difference includes both the representation and the resulting selected hyperparameters.

10.2 Future work

A direct next step would be to test binary cycle encoding on an additional predeclared dataset using the same saved nested protocol and a clearly separated final evaluation. Such work should not be described as completed here. Additional datasets with varied graph sizes, label balance, and cycle structure could assess whether the MUTAG result is dataset-specific. Incorporating edge-label-aware features would test whether bond categories add information beyond atom labels and topology.

A factorial follow-up could compare fixed selected parameters and nested retuning to separate representation effects from selection changes. Alternative classifiers could reveal whether binary encoding or λ scaling behaves differently outside axis-aligned trees. Formal uncertainty analyses should respect repeated-sample dependence, for example by using an appropriate repeated-CV procedure or an independent external test cohort. Any such extension would require new experiments; none is included in the present report.

11 Conclusion

This case study implemented WL, P-WL, and P-WL-C graph representations and evaluated two custom cycle-feature adaptations under nested cross-validation. The corrected established-method experiments completed 100 outer folds on MUTAG and 100 on PTC-MR. The binary cycle-presence follow-up completed 100 MUTAG folds; it was not evaluated on PTC-MR. Saved predictions and fold definitions support metric recomputation, paired comparisons, and result provenance.

WL achieved the highest MUTAG mean accuracy and macro-F1. P-WL-C improved over P-WL on MUTAG but remained below WL; on PTC-MR, P-WL-C was lower than P-WL. Positive λ values yielded identical predictions across the 100 tested folds per dataset, while $\lambda=0$ matched P-WL. Binary encoding changed predictions but reduced MUTAG accuracy and macro-F1 relative to P-WL-C. These results do not support a universal cycle-feature advantage or a positive binary-modification claim.

The project meets the methodological goals of implementing literature-based variants, modifying a representation, and measuring the modification's effect. The binary modification is custom and tested; its observed effect is negative. Negative experimental results are legitimate research findings when the method, scope, protocol, and limitations are reported transparently. Further claims of improvement or generality would require additional datasets and experiments.

References

[1] N. Shervashidze, P. Schweitzer, E. J. van Leeuwen, K. Mehlhorn, and K. M. Borgwardt, “Weisfeiler-Lehman Graph Kernels,” Journal of Machine Learning Research, vol. 12, no. 77, pp. 2539–2561, 2011. Available: https://www.jmlr.org/papers/v12/shervashidze11a.html. Local paper: references/wl-paper.pdf and references/wl-paper.txt.

[2] B. Rieck, C. Bock, and K. Borgwardt, “A Persistent Weisfeiler-Lehman Procedure for Graph Classification,” in Proceedings of the 36th International Conference on Machine Learning, Proceedings of Machine Learning Research, vol. 97, pp. 5448–5458, 2019. Available: https://proceedings.mlr.press/v97/rieck19a.html. Local paper: references/paper.pdf and references/paper.txt.

[3] K. M. Borgwardt Laboratory, “P-WL: A Persistent Weisfeiler–Lehman Procedure for Graph Classification,” authors’ source repository. Snapshot identity and hashes are retained in references/repository-head.txt and references/source_hashes.json. Accessed for implementation details; not treated as a separate novelty claim.

[4] C. Morris et al., “TUDataset: A collection of benchmark datasets for learning with graphs,” maintained dataset catalogue and bibliography. https://chrsmrrs.github.io/datasets/ and https://chrsmrrs.github.io/datasets/docs/bibliography/. The local original archives and their SHA-256 values are recorded in results/full/MUTAG_data.json, results/full/PTC_MR_data.json, and results/binary-cycle/dataset_statistics.json.

[5] A. K. Debnath, R. L. Lopez de Compadre, G. Debnath, A. J. Shusterman, and C. Hansch, “Structure-activity relationship of mutagenic aromatic and heteroaromatic nitro compounds,” Journal of Medicinal Chemistry, vol. 34, no. 2, pp. 786–797, 1991.

[6] C. Helma, R. D. King, S. Kramer, and A. Srinivasan, “The Predictive Toxicology Challenge 2000–2001,” Bioinformatics, vol. 17, pp. 107–108, 2001.

[7] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. The software version used in this project is recorded in the requirements and run metadata.

[8] M. Fey and J. E. Lenssen, “Fast Graph Representation Learning with PyTorch Geometric,” in ICLR Workshop on Representation Learning on Graphs and Manifolds, 2019. PyG TUDataset is used for loader verification; raw archive identity checks are described in data.py and the saved dataset metadata.

Reproducibility record

Final baseline predictions and aggregates: results/full/MUTAG_folds.csv, PTC_MR_folds.csv, MUTAG_summary.csv, and PTC_MR_summary.csv. Binary follow-up and paired records: results/binary-cycle/MUTAG_folds.csv, MUTAG_summary.csv, paired_folds.csv, paired_repeat_means.csv, comparison.json, verification.json, and preservation_verified.json. Methods and protocol: wl.py, pwl.py, data.py, evaluate.py, binary_cycle.py, README.md, NOTES.md, RESULTS.md, and SLIDES.md. These paths identify the specific project evidence behind the reported claims.